Will this website, other pages, and documentation be helpful to you?
Yes
Tell us more about what you found helpful or not helpful
Submit

Replicated Actor Properties

Property definitions, conditions and rules, property initialization and replication notifications.

Your Unreal Engine class might have properties that need to replicate in your multiplayer game. Unreal Engine actors maintain a list of properties with the `Replicated` and `ReplicatedUsing` variable specifiers. Unreal Engine's Replication System will check these properties for changes and replicate them to connected clients when changes occur. Unreal Engine's Replication System also provides a special callback function when a replicated property changes in value. Each client applies the updated value to its local version of the property.

This page covers the following topics:

- Replicating Actor Properties
- Property Initialization
- Customizing Conditional Replication
- Replicating Using Properties

Replicated Actor Properties

In general, you have two primary options to replicate actor properties:

- `Replicated`
- `ReplicatedUsing`

These are both property specifiers which are recognized by Unreal Engine's reflection system. `Replicated` properties provide generic replication, which is ideal for specific certain conditions that don't require custom properties. Properties with `ReplicatedUsing` provide notifications when a property changes, which is ideal for cases where you need to perform some action when a property changes.

Both of these specifiers are used in the header file with the `UPROPERTY()` specifier. This specifier allows Unreal Engine to keep track of the property, but it can be helpful to set a property within a replicated function, so it is also recommended to use `BlueprintReadWrite` when using `UPROPERTY()`.

Add a Replicated Property

Suppose you have an `Health` internal class member, `MaxHealth`, and this class has a property `Health`. You want to replicate `Health` to clients of an actor of this class.

In `MyCharacter.h`, add the `Replicated` metadata specifier to your `Health` property:

```
MyCharacter.h
```

1. Ensure that the `MyCharacter.h` is set to replicate within the derived actor's constructor:

```
MyCharacter.cpp
```

2. Override the `GetLifetimeReplicatedProps()` function and add a macro call to replicate the `Health` property to the lifetime of the derived actor instance:

```
MyCharacter.h
```

Add a Replicated Using Property

Suppose you have an `Health` internal class member, `MaxHealth`, and this class has a property `Health`. You want to replicate `Health` to clients of an actor of this class. You want to also perform some action when the `Health` property changes in value. In this instance, you must specify the name of the function that is called when the client when a property with the `ReplicatedUsing` specifier is replicated.

Suppose you have an `Health` internal class member, `MaxHealth`, and this class has a property `Health`. You want to also perform some action when the `Health` property changes in value. In this instance, you must specify the name of the function that is called when the client when a property with the `ReplicatedUsing` specifier is replicated.

1. In your `MyCharacter.h`, add the `ReplicatedUsing` metadata specifier to your `Health` property. In this instance, you must specify the name of the function that is called when the client receives a change to this replicated property. In this example, the function is `OnRep_Health`, followed by the property name `Health`:

```
MyCharacter.h
```

2. Ensure that the `MyCharacter.h` is set to replicate within the derived actor's constructor:

```
MyCharacter.cpp
```

3. Override the `GetLifetimeReplicatedProps()` function and add a macro call to replicate the `Health` property to the lifetime of the derived actor instance:

```
MyCharacter.h
```

4. Implement the property's associated `RepNotify`:

```
MyCharacter.cpp
```

Usage of Parameters in RepNotify

The `RepNotify` function can accept a single parameter if the parameter is a property `RepNotify` that is the same type as the replicated property. If you have a parameter in a `RepNotify`, the callback function will automatically fire on the previous value of the replicated property into the `RepNotify` call. To pass the previous replicated property value to the `OnRep_` function for the `RepNotify` example, use the following:

```
MyCharacter.h
```

```
MyCharacter.cpp
```

```
MyCharacter.h
```

```
MyCharacter.cpp
```

```
MyCharacter.h
```

```
MyCharacter.cpp
```

Differences with RepNotify in C++ and Blueprint

`RepNotifies` are different in Blueprint than they are in C++. In Blueprint, a `OnRep` node is a `Replicated` variable, and it is not a function. In C++, it is a function. Functions in Blueprint, such as `Begin Play`, `Event Tick`, `Event Hit`, `Event Overlap`, and `Event Destroyed`, are not replicated. Instead, the `RepNotifies` are used as the primary method of replicated property notifications for the same set of properties. Unreal Engine 4 does not support `RepNotifies` in Blueprint.

Add a Net Replicated Property

As previously mentioned, the `NotReplicated` specifier is useful for marking a property not to replicate. Alternatively, the `AdvancedDisplay` meta specifier provides an example of a net replicated property within a replicated struct.

```
DerivedActor.h
```

```
DerivedActor.cpp
```

Conditional Replication

Once you register a property for replication, you can't unregister it. Properties you set for replication will always replicate. However, you can use the `DOREPLIFETIME_CONDITION` macro to specify a condition for a property to replicate on some actors, according to the flags set on the actor and properties, thereby saving compression time.

To perform their control over how a property replicates, you can use replication conditions. By default, every replicated property only replicates on the condition that it has changed. Properties that have not changed do not replicate. Properties that have changed, but have the same value as the

Add a Replication Condition

To add a replication condition to your replicated properties, you can add a parameter/replication condition. To add a replication condition to your replicated property, use the `DOREPLIFETIME_CONDITION` macro:

```
DerivedActor.cpp
```

Notice that this example uses the `DOREPLICATE_CONDITION` macro instead of the `DOREPLICATE` macro since the replicated property has a condition.

> Replication condition works for properties that are replicated using both the `Replicated` and the `ReplicatedUsing` specifiers.

Add a RepNotify Condition

`RepNotifies` add an additional layer of customization over `Replicated` properties. You can set a `RepNotify` to only fire when a property changes. Unreal Engine's Replication System checks if the property is replicated or not, when the property changes. To add a `RepNotify` condition, use the `DOREPLICATE_CONDITION_NOTIFY` macro:

```
DerivedActor.cpp
```

Notice that this example uses the `DOREPLICATE_CONDITION_NOTIFY` macro instead of the `DOREPLICATE` macro.

> If you set your conditions inside your replication, you will be overwritten. `DOREPLICATE_EVER_CHANGES` resets to exactly `ReplicationCondition`, when `ReplicationCondition` is set to `DOREPLICATE_CHANGED`, the default behavior of replication for the function should be assumed of current header.

Replication Condition Reference

The table below contains a list of property replication conditions:

Condition	Description
`COND_None`	No conditions and replicates properties if changed.
`COND_InitialOnly`	Only attempts to replicate on the initial bunch.
`COND_OwnerOnly`	Only replicates to the ownin connection.
`COND_SkipOwner`	Replicates to every connection except the owner's owner.
`COND_SimulatedOnly`	Simulated actors.
`COND_AutonomousOnly`	Only replicates to autonomous actors.
`COND_SimulatedOrPhysics`	Replicates to simulated or `bRepPhysics` actors.
`COND_InitialOrOwner`	Replicates on the initial bunch, or to the ownin's owner.
`COND_Custom`	Custom condition, but like the ability to toggle on or off the Custom Property Replicator for more information.
`COND_ReplayOrOwner`	Only replicates to the replay connection, or to the ownin's owner.
`COND_ReplayOnly`	Only replicates to the replay connection.
`COND_SimulatedOrPhysicsOrReplay`	Replicates to simulated or `bRepPhysics` actors, or to replay connection.
`COND_InitialOrReplay`	Does not replicate to the replay connection.
`COND_Dynamic`	Checks the condition at runtime. Replicate is always replicate unless overwrite it in a new condition.
`COND_Never`	Never replicates.

Custom Property Replication

For their control over what and when properties replicate, you can use the `COND_Custom` replication condition. Unreal Engine's Replication System checks the `COND_Custom` condition, and if it is set, it checks the custom property replicator with a custom condition. This replicator is defined per actor, not per connection. As a result, you can use this to create a custom flag per change per connection to control the replication.

Add a Custom Replication Condition

To add a custom replication condition, follow these steps:

1. Follow the first two steps in the Add a Replicated Property section.
2. Override the `GetLifetimeReplicatedProps` function and add a macro call to replicate the `Health` property for the lifetime of the derived actor instance with the `COND_Custom` replication condition:

DerivedActor.h

```
#pragma once

#include "ReviveActor.generated.h"

UCLASS()
class AReviveActor : public AActor
{
  GENERATED_BODY()

  public:
    UPROPERTY(Replicated)
    int32 Health;

    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifeti

    // Derived Actor constructor
    AReviveActor(const class FPostConstructInitializeProperties & PCIP);
};
```

Copy full snippet

DerivedActor.cpp

```
#include "ReviveActor.h"
#include "Net/UnrealNetwork.h"

AReviveActor::AReviveActor(const class FPostConstructInitializeProperties & PCI
{
  bReplicates = true;
}

void AReviveActor::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLife
{
  // Call the Super
  Super::GetLifetimeReplicatedProps(OutLifetimeProps);

  // Add properties to replicated for the derived class
  DOREPLIFETIME_CONDITION(AReviveActor, Health, COND_Custom);
}
```

Copy full snippet

3. Define the property or function that provides a boolean expression for your custom replication condition override:

DerivedActor.h

```
#pragma once

#include "ReviveActor.generated.h"

UCLASS()
class AReviveActor : public AActor
{
  GENERATED_BODY()

  public:
    UPROPERTY(Replicated)
    int32 Health;

    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLife

    // Derived Actor constructor
    AReviveActor(const class FPostConstructInitializeProperties & PCIP);

    // Custom Replication Condition override function
    bool IsInvincible();
};
```

Copy full snippet

DerivedActor.cpp

```
#include "ReviveActor.h"
#include "Net/UnrealNetwork.h"

AReviveActor::AReviveActor(const class FPostConstructInitializeProperties & PCI
{
  bReplicates = true;
}

void AReviveActor::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLife
{
  // Call the Super
  Super::GetLifetimeReplicatedProps(OutLifetimeProps);

  // Add properties to replicated for the derived class
  DOREPLIFETIME_CONDITION(AReviveActor, Health, COND_Custom);
}

bool IsInvincible()
{
  bool bIsInvincible = false;

  // Custom logic to determine invincibility...

  return bIsInvincible;
}
```

Copy full snippet

4. Register your custom replication condition override in your derived class `PreReplication` function:

DerivedActor.h

```
#pragma once

#include "ReviveActor.generated.h"

UCLASS()
class AReviveActor : public AActor
{
  GENERATED_BODY()

  public:
    UPROPERTY(Replicated)
    int32 Health;

    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLife

    // Derived Actor constructor
    AReviveActor(const class FPostConstructInitializeProperties & PCIP);

    virtual void PreReplication(IRepChangedPropertyTracker& ChangedPropertyTra

    // Custom Replication Condition override function
    bool IsInvincible();
};
```

Copy full snippet

DerivedActor.cpp

```
#include "ReviveActor.h"
#include "Net/UnrealNetwork.h"

AReviveActor::AReviveActor(const class FPostConstructInitializeProperties & PCI
{
  bReplicates = true;
}

void AReviveActor::GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLife
{
  // Call the Super
  Super::GetLifetimeReplicatedProps(OutLifetimeProps);

  // Add properties to replicated for the derived class
  DOREPLIFETIME_CONDITION(AReviveActor, Health, COND_Custom);
}

void AReviveActor::PreReplication(IRepChangedPropertyTracker& ChangedPropertyTra
{
  // Function where the Custom condition is implemented.
  */
  void AReviveActor::PreReplication(IRepChangedPropertyTracker& ChangedPropertyTra
  {
    // In this case, a function IsInvincible()
    // If the actor is invincible, don't replicate health
    bool bIsInvincible = IsInvincible();
    DOREPLIFETIME_ACTIVE_OVERRIDE(AReviveActor, Health, (!bIsInvincible));
  }

  bool IsInvincible()
  {
    bool bIsInvincible = false;

    // Custom logic to determine invincibility...

    return bIsInvincible;
  }
```

Copy full snippet

`Health` now only replicates as long as `IsInvincible()` returns false.

A custom replication condition can do anything that the predefined replication conditions can and more, so why not use custom conditions all the time?

- **Speed**: Custom replication conditions take more resources and more time.
- **Connection**: Custom replication conditions cannot change on a per-connection basis.

Custom Property Replication Macros

There are a few custom property replication macros that have a few differences that you should be aware of. All of these macros use the function `FRepPropertyListInitMemberOverrides`. These macros all build an `FRepPropertyDescriptor` struct from the given macro parameters. The difference between the macros is when the descriptor struct is built and whether or not it works for replicated arrays. The following table describes the differences between these macros:

Macro	Compile Time or Runtime	Works with Arrays
`DOREPLIFETIME_ACTIVE_OVERRIDE`	Runtime	Yes
`DOREPLIFETIME_ACTIVE_OVERRIDE_FAST`	Compile time	No
`DOREPLIFETIME_ACTIVE_OVERRIDE_FAST_STATIC_ARRAY`	Compile time	Yes

> The `_FAST` macro versions build the replication property descriptor struct at compile time. This saves CPU cycle time since the replication system does not need to search for the provided `UPROPERTY`, and then build the struct at runtime.

Replicate Object References

Replication of object references is automatically handled by Unreal Engine's replication system. If you have a `UObject` property that is replicated, references to that object are sent over network connections as a `FNetworkGUID`, that is assigned by the authoritative server. The server then notifies all connected clients of this assignment.

Add a Replicated Object Reference

To replicate an object reference, mark a `UObject` property as replicated:

DerivedActor.h

```
#pragma once

#include "ReviveActor.generated.h"

UCLASS()
class AReviveActor : public AActor
{
  GENERATED_BODY()

  UPROPERTY(Replicated)
  AActor* Owner;

  // Derived Actor constructor
  AReviveActor(const class FPostConstructInitializeProperties & PCIP);
};
```

Copy full snippet

The Owner property is a replicated reference to the actor that this property refers to.

Guidelines For Network Object References

For an object to be legally referenced over the network, it must be supported for networking. To check if an object is supported for networking, call `UObject::IsSupportedForNetworking()`.

The following objects can have references to them replicated over a network connection:

- Replicated actors
- Stably-named, non-replicated actors
- Replicated components
- Stably-named, non-replicated actor components
- Any non-actor, non-component UObject from a loaded package

Stably Named Objects

Stably Named Objects are objects that exist both on the server and the client, and have the same name on both. Actors are stably named if they were loaded directly from packages, not spawned during gameplay.

Actor components are stably named if:

- Loaded directly from a package
- Added with simple construction scripts
- Manually marked via `UActorComponent::SetNetAddressable`.
  - Only use this when you know you are manually naming the actor component so it has the same name on the server and client. A good example of this are components added in a `AActor` C++ constructor.

Property Replication Reference

The following table describes property replication macros available in Unreal Engine, in addition to:

- The function to call the macro from.
- Whether the associated `FRepPropertyDescriptor` is constructed at compile time or runtime.
- Whether the macro works with arrays.

Macro	Description	Where to Call Macro	Compile Time or Runtime	Works with Arrays
`DOREPLIFETIME`	Replicate a property with no customization.	`GetLifetimeReplicatedProps`	Runtime	Yes
`DOREPLIFETIME_WITH_PARAMS`	Replicate a property with an `FDoRepLifetimeParams` struct to control how it is replicated.	`GetLifetimeReplicatedProps`	Runtime	Yes
`DOREPLIFETIME_WITH_PARAMS_FAST`	Replicate a property with an `FDoRepLifetimeParams` struct to control how it is replicated.	`GetLifetimeReplicatedProps`	Compile Time	No
`DOREPLIFETIME_WITH_PARAMS_FAST_STATIC_ARRAY`	Replicate a property with an `FDoRepLifetimeParams` struct to control how it is replicated.	`GetLifetimeReplicatedProps`	Compile Time	Yes
`DOREPLIFETIME_CONDITION`	Replicate a property with an `ELifetimeRepCondition`.	`GetLifetimeReplicatedProps`	Runtime	Yes
`DOREPLIFETIME_CONDITION_NOTIFY`	Replicate a property with a `ELifetimeRepNotifyCondition`.	`GetLifetimeReplicatedProps`	Runtime	Yes
`DOREPLIFETIME_ACTIVE_OVERRIDE`	Replicate a property with a custom condition. Must be accompanied by a call to `DOREPLIFETIME_CHANGED_PROPERTY_FAST(...)`.	`PreReplication`	Runtime	Yes
`DOREPLIFETIME_ACTIVE_OVERRIDE_FAST`	Replicate a property with a custom condition. Must be accompanied by a call to `DOREPLIFETIME_CHANGED_PROPERTY_FAST(...)`.	`PreReplication`	Compile Time	No
`DOREPLIFETIME_ACTIVE_OVERRIDE_FAST_STATIC_ARRAY`	Replicate a property with a custom condition. Must be accompanied by a call to `DOREPLIFETIME_CHANGED_PROPERTY_FAST(...)`.	`PreReplication`	Compile Time	Yes

> You can find the definitions for these macros in Engine/Source/Runtime/Engine/Public/Net/UnrealNetwork.h :

`programming` `networking` `properties` `DoreplifetimeNotify` `repnotify` `conditions` `referenced`

Ask questions and help your peers Developer Forums

Write your own tutorials or read those from others Learning Library

Collapse